



# MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

---

## UNIT-1

**Dr. Annu Sharma**  
**Associate Professor**

Department of Computer Applications

# MOBILE APP DEVELOPMENT

---

## UNIT - 1

### Introduction to Mobile App Development

- Mobile application development is the set of processes and procedures involved in writing software for small, wireless computing devices, such as smartphones and other hand-held devices.

### A. Android

**Developer: Google**

**Launch Year: 2008**

**Latest Version: Android 14**

**App Store: Google Play Store**

**Devices: ???**



## **B. iOS**

**Developer: Apple Inc.**

**Launch Year: 2007**

**Latest Version: iOS 17**

**App Store: Apple Play Store**

**Devices: ???**



### C. Windows Mobile/ Phone

**Developer: Microsoft**

**Launch Year: 2000/ 2010**

**Latest Version: Discontinued from 2019**

**App Store: Microsoft Store**

**Devices: ???**



## **D. Blackberry OS**

**Developer: Blackberry Limited**

**Launch Year: 1999**

**Latest Version: Discontinued from 2013**

**App Store: BlackBerry World**

**Devices: ???**



# Mobile App Development

## Mobile Operating Systems

---



### Others:

- ☐ **Palm OS**
- ☐ Symbian OS
- ☐ Harmony OS
- ☐ KaiOS
- ☐ Tizen
- ☐ FirefoxOS
- ☐ Ubuntu OS
- ☐ Sailfish OS
- ☐ MeeGo OS



## **Alternatives for Building Mobile Apps**

There are four major development approaches when building mobile applications

- **Native Mobile Applications**
- **Cross-Platform Native Mobile Applications**
- **Hybrid Mobile Applications**
- **Progressive Web Applications**

## **Native mobile applications**

**They are written in the programming language and frameworks provided by the platform owner and running directly on the operating system of the device such as iOS and Android.**

## **Hybrid mobile applications**

**Cross-platform native mobile applications can be written in variety of different programming languages and frameworks, but they are compiled into a native application running directly on the operating system of the device.**

### Web mobile applications

Hybrid mobile applications are built with standard web technologies - such as JavaScript, CSS, and HTML5 - and they are bundled as app installation packages. Contrary to the native apps, hybrid apps work on a 'web container' which provides a browser runtime and a bridge for native device APIs via Apache Cordova.

### Progressive Web Applications

PWAs offer an alternative approach to traditional mobile app development by skipping app store delivery and app installations. PWAs are web applications that utilize a set of browser capabilities - such as working offline, running a background process, and adding a link to the device home screen - to provide an 'app like' user experience.

# MOBILE APP DEVELOPMENT

---

## Introduction to Hybrid Mobile App Development



- Hybrid mobile apps are applications designed to work across multiple platforms (iOS, Android, etc.) using a single codebase.
- They combine elements of both native and web applications.
- A hybrid mobile app is built using web technologies (HTML, CSS, JavaScript) and is wrapped in a native container.
- This approach allows the app to be deployed on multiple platforms from a single codebase while still accessing native device features.

# Mobile App Development

## Comparing Native and Hybrid Mobile Apps

| Feature                  | Native App  | Hybrid App  |
|--------------------------|-------------|------------------------------------------------------------------------------------------------|
| Development Language     | Native only | Native and web / web only                                                                      |
| Device Access            | Complete    | Complete                                                                                       |
| Device Specific Features | High        | Moderate                                                                                       |
| Speed                    | Very fast   | Medium                                                                                         |
| App Store                | Available   | Available                                                                                      |
| Approval Process         | Mandatory   | Low overhead                                                                                   |
| Code Portability         | None        | High                                                                                           |

# Mobile App Development

## Comparing Native and Hybrid Mobile Apps

| Feature               | Native App | Hybrid App <input type="checkbox"/> |
|-----------------------|------------|-------------------------------------|
| Advanced Graphics     | High       | Moderate                            |
| UI / UX               | High       | Moderate                            |
| Access to Native APIs | High       | Moderate                            |
| Development Cost      | Expensive  | Reasonable                          |

# Mobile App Development

## Hybrid Mobile App Development Framework



Adobe PhoneGap



Ionic



Xamarin

Intel® XDK

Intel XDK



React Native



Flutter



Corona SDK



Mobile Angular UI



Native Scripts



jQuery Mobile



Framework7



Appcelerator Titanium



Sencha Touch



Swiftic



Apache Cordova



# MOBILE APP DEVELOPMENT

---

## Getting Started with DART



- **Dart is an open-source** general-purpose programming language developed by Google.
- It supports application development on both the client and server side
- It is widely used for the **development of Android apps, iOS apps, IoT**(Internet of Things), and web applications using the Flutter Framework.
- Syntactically, Dart bears a strong resemblance to Java, C, and JavaScript.
- It is a **dynamic object-oriented language** with closure and lexical scope.
- The Dart language was released in 2011 but came into popularity after 2015 with Dart 2.0.

## Why Use Dart

- **Fast & Smooth:** Dart compiles to native code for speedy performance, ideal for mobile apps.
- **Easy to Learn:** Similar to familiar languages like Java or Javascript, making it approachable for new developers.
- **Flutter Power:** Dart is the heart of Flutter, a popular framework for building beautiful and functional mobile apps.
- **One Code, Many Places:** Develop for mobile, web, and even desktop with a single codebase (primarily with Flutter).

# MOBILE APP DEVELOPMENT

---

## Getting Started with Flutter Framework



- Flutter is an open-source UI toolkit developed by Google for building natively compiled applications across mobile, web, and desktop from a single codebase.
- It allows developers to create high-performance, visually appealing apps with a rich set of pre-designed widgets and tools.
- Flutter uses Dart, a programming language developed by Google.
- Dart is optimized for building user interfaces and provides features like hot reload, strong typing, and asynchronous programming, which are beneficial for UI development.



THANK YOU

---

**Dr. Annu Sharma**  
**Associate Professor**

Department of Computer Applications



# **MOBILE APP DEVELOPMENT**

**(UQ23CA243AA1)**

---

**Ms. Annu Sharma**  
**Associate Professor**

Department of Computer Applications

# MOBILE APP DEVELOPMENT

---

## Introduction to Dart Programming Variables

### Data types – Numeric, String

**Dr. Annu Sharma**

Department of Computer Applications



# Mobile App Development

## Introduction to Dart Programming

---



- Dart is an object-oriented programming language developed by Google.
- Primarily used for building web, server, and mobile applications.
- It is best known for its use in conjunction with Flutter, Google's UI toolkit for building natively compiled applications.

- In Dart, the “*main*” function is the entry point of a Dart application.
- It is the starting point of execution and where the Dart runtime begins when running a Dart program.
- Here's a detailed look at the main function:

```
void main()  
{  
    // Code to execute  
}
```

**void:** The return type of the main function is void, meaning it does not return any value.

**main:** The function name must be main()  
The function does not take any parameters in its most basic form.

```
void main()  
{  
    print('Hello, Dart !');  
}
```

- Variables are one of the basics and Dart comes with support for type inference.
  - `var codeno = 20;`
  - `var myName = "Dart";`
    - `codeno` is an **integer** while `myName` is a **string**.

- Dart can infer the type of the variable by looking at the value that has been assigned.
- In other words, the Dart compiler is smart enough to figure out by itself which is the correct type of the variable.
- The variables can also be declared as
  - `int codeno = 20;`
  - `String myName = "Dart"`

- `dynamic codeno = 20;`
- `dynamic myName = "Dart"`
- dynamic keyword can be used with variable having any type.
- The type of a dynamic variable is evaluated at runtime and thus, for a proper usage, it is often **not used**.
- When a variable need not be initialized immediately, use the late keyword.
  - `late String myName;`

- A variable declared as *final* can be set only once.
- Changing its content later will result in an error.
  - `final myName = "Dart";`

- Prefer initializing variables with *var* as much as possible
- When the type is not so easy to guess or identify, initialize it explicitly to increase the readability of the code.
- Use Object or dynamic only if it's really needed but it's almost never the case.
- Variables can be declared as Implicit, Explicit and Late



# Mobile App Development

## Dart Data Types

---



- Numeric – Integer and Double
- String
- Boolean
- Enum
- List
- Map
- Set

- Dart has two type of numeric data
  - **Integer - int**
    - 64-bit at maximum, integer values.
    - This type ranges from  $-2^{63}$  to  $2^{63}-1$ .
  - **Double - double**
    - 64-bit double-precision floating point numbers that follow the classic IEEE 754 standard definition.

- Both *double* and *int* are subclasses of numeric which provides many useful methods such as:
  - `parse(string)`,
  - `abs()`,
  - `ceil()`,
  - `toString()..`

# Mobile App Development

## Dart Data Types - Numeric

---



- `var a = 1; // int`
- `var b = 1.0; // double`
- `int x = 8;`
- `double y = b + 6;`

- A string is an ordered sequence of UTF-16 values surrounded by either single or double quotes.
  - `var str1 = "Double quoted";`
  - `var str2 = 'Single quoted';`
    - `// No differences between str1 and str2`
  - `var age = 20;`
  - `var myAge = "I am $age years old";`
    - `// Interpolate an integer into a string`

# Mobile App Development

## Dart Data Types - String

---



- `var str1 = "${25.abs()}"`
  - `// Expressions need '{' and '}' preceded by $`
- `${expr}` is a shorthand to call the `toString()` method.
- A string can be either single or multiline.
  - Single line strings are shown above using single or double quotes.
  - Multiline strings are written using triple quotes.

```
void main()  
{  
    var string = 'Dart is fun';  
    var newString = string.substring(0, 5);  
    print(newString);  
}
```

You can use the plus (+) operator to concatenate strings:

```
'Dart ' + 'is ' + 'fun!'; // 'Dart is fun!'
```

You can also use adjacent string literals for concatenation:

```
'Dart ' 'is ' 'fun!'; // 'Dart is fun!'
```

You can use `${}` to interpolate the value of Dart expressions within strings.

The curly braces can be omitted when evaluating identifiers:

```
string = 'dartlang'; '$string has ${string.length} letters'; // 'dartlang has 8 letters'
```



A string is represented by a sequence of Unicode UTF-16 code units accessible through the codeUnitAt or the codeUnits members:

```
string = 'Dart'; string.codeUnitAt(0); // 68  
string.codeUnits;
```

The string representation of code units is accessible through the index operator:

```
string[0]; // 'D'
```

**1. What data type would you use in Dart to store true or false values?**

- A) int
- B) bool
- C) String
- D) dynamic

**2. In Dart, which data type is used to represent decimal numbers?**

- A) int
- B) double
- C) num
- D) var

**3. What data type does Dart use to represent a sequence of characters?**

- A) String
- B) List
- C) Map
- D) Set



THANK YOU

---

**Dr. Annu Sharma**

**Associate**

**Professor**

Department of Computer Science



# **MOBILE APP DEVELOPMENT**

**(UQ23CA243AA1)**

---

**Dr. Annu Sharma**  
**Associate Professor**

Department of Computer Applications

# MOBILE APP DEVELOPMENT

---

**Data types – Enum, Boolean**  
**Dart Comments**

**Dr. Annu Sharma**

Department of Computer Applications

- Also known as "enums".
- They are containers for constant values that can be declared with the enum keyword.
- Each item of the enum has an associated number, called index, which corresponds to the zero-based position of the value in the declaration.
- This number can be accessed by using the index property.

# Mobile App Development

## Dart Data Types – Enum Example

---



```
enum Color { Red, Green, Blue, Black, Orange, Pink }
```

```
void main()  
{  
    Color selectedColor = Color.Black;  
    print(selectedColor);  
}
```

# Mobile App Development

## Dart Data Types – Example 2



```
enum Fruits { Apple, Pear, Grapes, Banana, Orange }
```

```
void main() {
```

```
    var a = Fruits.Apple.index; // 0
```

```
    var b = Fruits.Pear.index; // 1
```

```
    var c = Fruits.Grapes.index; // 2
```

```
    print(a);  print(b);  print(c);
```

```
}
```



- Assign to the bool type only the literals true or false, which are both compile-time constants.
  - `bool test = 5 == 0; // false`
  - `bool test2 = !test; // has the opposite value of test`
  - `var oops = 0.0 / 0.0; // evaluates to 'Not a Number' (NaN)`
  - `bool didIFail = oops.isNaN;`

# Mobile App Development

## Dart Basic Example



```
void main()
{
    // Declare variables

    String name = "Annu";
    int empid = 007;
    double height = 5.3;
    bool isActive = true;

    // Print the variables

    print("Name: $name");
    print("EmpCode: $empid");
    print("Height: $height");
    print("Active: $isActive");
}
```

1. **Dart Single line Comment:** Dart single line comment is used to comment a line until line break occurs. It is done using a double forward-slash (//).

Example:

```
// This is a single line comment.
```

2. **Dart Multi-Line Comment:** Dart Multiline comment is used to comment out a whole section of code. It uses ‘/\*’ and ‘\*/’ to start and end a multi-line comment, respectively.

Example:

```
/* These  
are  
multiple  
line of  
comments */
```

**3. Dart Documentation Comment:** Dart Documentation Comments are a special type of comment used to provide references to packages, software, or projects. Dart supports two types of documentation comments “///”(C# Style) and “/\*\*.....\*/”(JavaDoc Style).

Example:

```
/// This is
```

```
/// a documentation
```

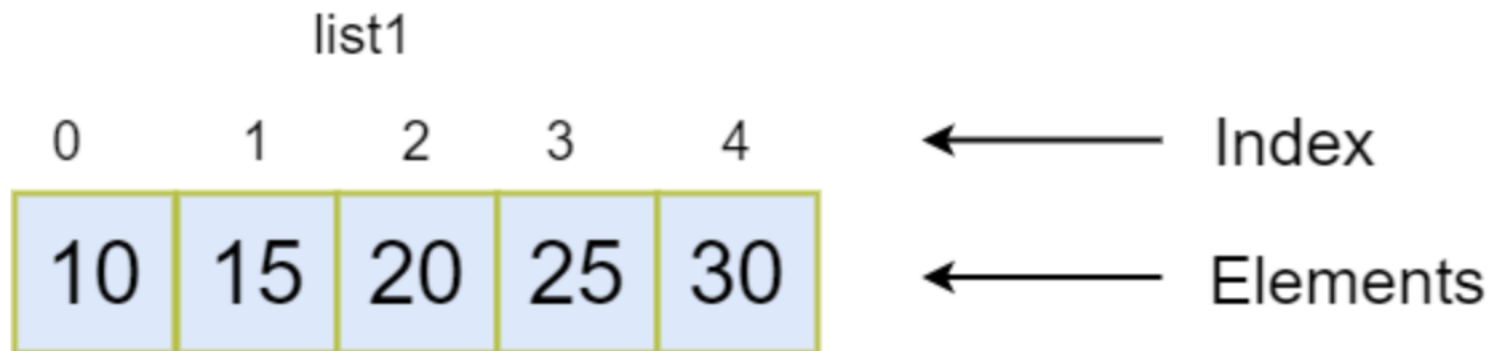
```
/// comment
```

# Mobile App Development

## Dart Data Types - List

The Dart list is defined by storing all elements inside the square bracket ([]) and separated by commas (,).

```
var list1 = [10, 15, 20, 25, 25]
```



## Types of Lists

The Dart list can be categorized into two types –

- Fixed Length List
- Growable List

### Fixed Length List

The fixed-length lists are defined with the specified length. We cannot change the size at runtime.

### Syntax - Create the list of fixed-size

```
1.var list_name = new List(size)
```

### Syntax - Initialize the fixed size list element

```
1.list_name[index] = value;
```

### Example of Fixed Length LIST

```
void main() {  
  var list1 = new List(5);  
  list1[0] = 10;  
  list1[1] = 11;  
  list1[2] = 12;  
  list1[3] = 13;  
  list1[4] = 14;  
  
  print(list1);  
}
```

#### Output:

[10, 11, 12, 13, 14]



### Growable List

The list is declared without specifying size is known as a Growable list. The size of the Growable list can be modified at the runtime. The syntax of the declaring Growable list is given below.

#### Syntax - Declaring a List

```
// creates a list with values
```

```
var list_name = [val1, val2, val3]
```

Or

```
// creates a list of the size zero
```

```
var list_name = new List()
```

#### Syntax - Initializing a List

```
list_name[index] = value;
```

# Mobile App Development

## Dart Data Types - List



### Example – 1

```
void main() {  
    var list1 = [10,11,12,13,14,15];  
    print(list1);  
}
```

### Output:

[10, 11, 12, 13, 14, 15]

### Example - 2

```
void main() {  
  
    var list1 = <int>[];  
    list1.add(10);  
    list1.add(11);  
    list1.add(12);  
    list1.add(13);  
    print(list1);  
}
```

### Output:

[10, 11, 12, 13]

# Mobile App Development

## Dart Data Types – List Properties

| Property   | Description                                                |
|------------|------------------------------------------------------------|
| first      | It returns the first element case.                         |
| isEmpty    | It returns true if the list is empty.                      |
| isNotEmpty | It returns true if the list has at least one element.      |
| length     | It returns the length of the list.                         |
| last       | It returns the last element of the list.                   |
| reversed   | It returns a list in reverse order.                        |
| Single     | It checks if the list has only one element and returns it. |

### Inserting Element into List

Dart provides four methods which are used to insert the elements into the lists. These methods are given below.

- add()
- addAll()
- insert()
- insertAll()

#### Syntax -

```
list_name.add(element);
```

#### Example -

```
void main() {  
    var odd_list = [1,3,5,7,9];  
    print(odd_list);  
    odd_list.add(11);  
    print(odd_list);  
}
```

#### Output:

```
[1, 3, 5, 7, 9] [1, 3, 5, 7, 9, 11]
```

### Syntax -

```
list_name.addAll([val1,val2,val3,?..valN]);
```

Let's understand the following example -

```
void main() {  
    var odd_list = [1,3,5,7,9]  
    print(odd_list);  
    odd_list.addAll([11,13,14]);  
    print(odd_list);  
}
```

### Output:

```
[1, 3, 5, 7, 9] [1, 3, 5, 7, 9, 11, 13, 14]
```

# Mobile App Development

## Dart Data Types - List



### The insert() Method

The **insert()** method provides the facility to insert an element at specified index position. We can specify the index position for the value to be inserted in the list. The syntax is given below.

```
list_name.insert(index,value);
```

Let's understand the following example -

```
void main(){  
    List lst = [3,4,2,5];  
    print(lst);  
    lst.insert(2,10);  
    print(lst);  
}
```

**Output:**

```
[3, 4, 2, 5] [3, 4, 10, 2, 5]
```

### The insertAll() Method

The insertAll() function is used to insert the multiple value at the specified index position. It accepts index position and list of values as an argument. The syntax is given below.

#### Syntax -

```
list_name.insertAll(index, iterable_list_of_value)
```

Let's understand the following example -

#### Example -

```
void main(){  
    List lst = [3,4,2,5];  
    print(lst);  
    lst.insertAll(0,[6,7,10,9]);  
    print(lst);  
}
```

#### Output:

```
[3, 4, 2, 5] [6, 7, 10, 9, 3, 4, 2, 5]
```

### Updating List

The Dart provides the facility to update the list and we can modify the list by simply accessing its element and assign it a new value. The syntax is given below.

#### **Syntax -**

```
list_name[index] = new_value;
```

Let's understand the following example -

#### **Example -**

```
void main(){  
    var list1 = [10,15,20,25,30];  
    print("List before updation: ${list1}");  
    list1[3] = 55;  
    print("List after updation:${list1}");  
}
```

#### **Output:**

List before updation: [10, 15, 20, 25, 30] List after updation: [10, 15, 20, 55, 30]



**replaceRange()** - The Dart provides **replaceRange()** function which is used to update within the given range of list items. It updates the value of the elements with the specified range. The syntax is given below.

### Syntax -

```
list_name.replaceRange(int start_val, int end_val, iterable);
```

Let's understand the following example -

### Example -

```
void main(){  
    var list1 = [10,15,20,25,30];  
    print("List before updation: ${list1}");  
    list1.replaceRange(0,4,[1,2,3,4]) ;  
    print("List after updation using replaceAll() function : ${list1}");  
}
```

### Output:

List before updation: [10, 15, 20, 25, 30] List after updation using replaceAll() function : [1, 2, 3, 4, 30]

## Removing List Elements

The Dart provides following functions to remove the list elements.

- remove()
- removeAt()
- removeLast()
- removeRange()

### The remove() Method

It removes one element at a time from the given list. It accepts element as an argument. It removes the first occurrence of the specified element in the list if there are multiple same elements. The syntax is given below.

#### Syntax -

```
list_name.remove(value)
```

Let's understand the following example -

#### Example -

```
void main(){  
    var list1 = [10,15,20,25,30];  
    print("List before remove element : ${list1}");  
    list1.remove(20) ;  
    print("List after removing element : ${list1}");  
}
```

#### Output:

List before remove element : [10, 15, 20, 25, 30] List after removing element : [10, 15, 25, 30]

### The removeAt() Method

It removes an element from the specified index position and returns it. The syntax is given below.

#### Syntax -

list\_name.removeAt(**int** index)

#### Example -

```
void main(){  
    var list1 = [10,11,12,13,14];  
    print("List before remove element : ${list1}");  
    list1.removeAt(3) ;  
    print("List after removing element : ${list1}");  
}
```

#### Output:

List before remove element : [10, 11, 12, 13, 14] List after removing element : [10, 11, 12, 14]

### The removeLast() Method

The removeLast() method is used to remove the last element from the given list. The syntax is given below.

#### Syntax-

```
list_name.removeLast()
```

Let's understand the following example.

#### Example -

```
void main(){  
    var list1 = [12,34,65,76,80];  
    print("List before removing element:${list1}");  
    list1.removeLast();  
    print("List after removed element:${list1}");  
}
```

#### Output:

List before removing element:[12, 34, 65, 76, 80] List after removed element:[12, 34, 65, 76]

### The `removeRange()` Method

This method removes the item within the specified range. It accepts two arguments - **start index** and **end index**. It eliminates all element which lies in between the specified range. The syntax is given below.

#### Syntax -

```
list_name.removeRange();
```

#### Example -

```
void main(){  
    var list1 = [12,34,65,76,80];  
    print("List before removing element:${list1}");  
    list1.removeRange(1,3);  
    print("List after removed element:${list1}");  
}
```

#### Output:

List before removing element:[12, 34, 65, 76, 80] List after removed element:[12, 76, 80]



THANK YOU

---

**Dr. Annu Sharma**

**Associate Professor**

Department of Computer Science



# **MOBILE APP DEVELOPMENT**

**(UQ23CA243AA1)**

---

**Dr. Annu Sharma**  
**Associate Professor**

Department of Computer Applications



# MOBILE APP DEVELOPMENT

---

## Dart Data types – Set

**Dr. Annu Sharma**

Department of Computer Applications

- The Dart Set is the unordered collection of the different values of the same type.
- Set doesn't allow storing the duplicate values. The set must contain unique values.
- It plays an essential role when we want to store the distinct data of the same type into the single variable.
- Once we declare the type of the Set, then we can have an only value of the same type. The set cannot keep the order of the elements.

# Mobile App Development

## Dart Data Types – Declaring Set



- Dart provides the two methods to declare/initialize an empty set.
- The set can be declared by using the {} **curly braces** preceded by a type argument or declare the variable type **Set** with curly braces {}.

### Syntax:

```
var setName = <type> {};
```

Or

```
Set<type> setname = {};
```

The '*setName*' refers to the name of the set variable, and '*type*' refers to the data type of the set.

# Mobile App Development

## Dart Data Types – Declaring Set



### Example:

```
void main(){  
    var weeknames = <String>{"Monday","Tuesday", "Wednesday","Thursday"};  
    print(weeknames);  
}
```

### Output:

{Monday,Tuesday, Wednesday,Thursday}

- The Dart provides the two methods **add()** and **addAll()** to insert an element into the given Set.
- **add()** method is used to add the single item into the given set one at a time.
- **addAll()** method is used to add the multiple elements to an existing set.

### Syntax:

Set\_name.add(<value>);

Or

Set\_name.addAll(val1,val2....valN)

### Example:

```
void main(){  
    var weeknames = <String>{"Monday", "Tuesday", "Wednesday", "Thursday"};  
    // Declaring empty set  
    var wk = <String>{};  
    wk.add("Monday");  
    print(wk);  
    // Adding multiple elements  
    wk.addAll(weeknames);  
    print(wk);  
}
```

### Output:

```
{Monday}  
{Monday, Tuesday, Wednesday, Thursday}
```

- Dart provides the **elementAt()** method, which is used to access the item by passing its specified index position.
- The set indexing starts from the 0 and goes up to size - 1, where size is the number of the element exist in the Set.
- It will throw an error if we enter the bigger index number than its size.

### Syntax:

`Set_name.elementAt(index)`

### Example -

```
void main(){  
    print("Access element from the Set");  
    var weeknames = {"Monday","Tuesday","Wednesday","Thursday"};  
    print(weeknames);  
    var x = weeknames.elementAt(3);  
    print(x);  
}
```

### Output:

Access element from the Set

{Monday, Tuesday, Wednesday, Thursday}

Thursday



- Dart provides the **contains()** method, which is used to find an element in the set.
- It accepts the single item as an argument and returns the result in Boolean type.
- If the given element is present in the set, it returns true otherwise false.

### Syntax:

```
set_name.contains(value);
```

### Example -

```
void main() {  
    print("Example - Find Element in the given Set");  
    var names = <String>{"Monday","Tuesday","Wednesday","Thursday"};  
    if(names.contains("Tuesday"))  
    {  
        print("Element Found");  
    }  
    else  
    {  
        print("Element not found");  
    } }  
}
```

### Output:

Example - Find Element in the given Set  
Element Found

- The **remove()** method is used to eliminate or remove an element from the given set.
- It takes the value as an argument; the value is to be removed in the given set.

### Syntax -

```
set_names.remove(value)
```

### Example -

```
void main() {  
    print("Example - Remove Element in the given Set");  
    var names = <String>{"Monday","Tuesday","Wednesday","Thursday"};  
    print("Before remove : ${names}");  
    names.remove(" Wednesday ");  
    print("After remove : ${names}");  
}
```

### Output:

Example - Remove Element in the given Set

Before remove : {Monday, Tuesday, Wednesday, Thursday}

After remove : {Monday, Tuesday, Thursday}

### Example -

```
void main() {  
    print("Example - Remove all elements in the given Set");  
    var names = <String>{"Monday","Tuesday","Wednesday","Thursday"};  
    print("Before remove : ${names}");  
    names.clear();  
    print("After remove : ${names}");  
}
```

### Output:

Example - Remove Element in the given Set

Before remove : {Monday, Tuesday, Wednesday, Thursday}

After remove : {}

- Dart Set provides to perform following set operations.
- **Union** - The union is set to combine the value of the two given sets a and b.
- **Intersection** - The intersection of the two-set a and b returns all elements, which is common in both sets.
- **Subtracting** - The subtracting of two sets a and b ( $a-b$ ) is the element of set b is not present in the set a.

### Example -

```
void main() {  
    var x = <int>{10,11,12,13,14,15};  
    var y = <int>{12,18,29,43};  
    var z = <int>{2,5,10,11,32};  
    print("Example - Set Operations");  
    print("x union y is -");  
    print(x.union(y));  
    print("x intersection y is - ");  
    print(x.intersection(y));  
    print("y difference z is - ");  
    print(y.difference(z)); }  
}
```

### Output:

Example - Set Operations

x union y is - {10, 11, 12, 13, 14, 15, 18, 29, 43}

x intersection y is - {12}

y difference z is - {12, 18, 29, 43}

# Mobile App Development

## Dart Data Types – Set Properties



| Property   | Description                                                |
|------------|------------------------------------------------------------|
| first      | It returns the first element case.                         |
| isEmpty    | It returns true if the list is empty.                      |
| isNotEmpty | It returns true if the list has at least one element.      |
| length     | It returns the length of the list.                         |
| last       | It returns the last element of the list.                   |
| reversed   | It returns a list in reverse order.                        |
| Single     | It checks if the list has only one element and returns it. |





THANK YOU

---

**Dr. Annu Sharma**  
**Associate Professor**  
Department of Computer Science



# **MOBILE APP DEVELOPMENT**

**(UQ23CA243AA1)**

---

**Dr. Annu Sharma**  
**Associate Professor**

Department of Computer Applications

# MOBILE APP DEVELOPMENT

---

Conditions in Dart

Looping in Dart

Break and Continue Statements

Switch Case Statement

**Annu Sharma**

Department of Computer Applications

You can use following conditions to control the flow of your program.

- **If Condition**
- **If-Else Condition**
- **If-Else-If Condition**
- **Switch case**

# Mobile App Development

## Control Flows – if-else-if condition



```
void main(){
    int num1 = 1500;
    int num2 = 1000;
    int num3 = 500;

    if(num1 > num2 && num1 > num3){
        print("Num 1 is greater: i.e $num1");
    }
    else if(num2 > num1 && num2 > num3){
        print("Num2 is greater: i.e $num2");
    }
    else if(num3 > num1 && num3 > num2){
        print("Num3 is greater: i.e $num3");
    }
}
```

### OUTPUT:

Num 1 is greater: i.e 1500

```
void main() {  
    var dayOfWeek = 2;  
    switch (dayOfWeek) {  
        case 1:  
            print("Day is Sunday.");  
            break;  
        case 2:  
            print("Day is Monday.");  
            break;  
        case 3:  
            print("Day is Tuesday.");  
            break;  
        default:  
            print("Invalid Weekday.");  
            break;  
    }  
}
```

**OUTPUT:**

Day is Monday.

```
void main() {  
    const weather = "grapes";  
    switch (weather) {  
        case "apple":  
            print("Red colored fruit.");  
            break;  
        case "orange":  
            print("Orange colored fruit.");  
            break;  
        case "mango":  
        case "banana":  
            print("Yellow colored fruit.");  
            break;  
        default:  
            print("No fruits found..");  
            break;  
    }  
}
```

**OUTPUT:**

No fruits found..

### Example using Break

## Break Statement

- The break statement **exits the nearest enclosing loop or switch statement immediately**.
- When Dart encounters break, it **stops executing the loop completely** and moves to the next statement after the loop.
- It is useful when you want to **stop looping based on a certain condition**.  
**Example use case:** Stop searching once you find the desired item.



### Example using Continue

#### Continue Statement

- The continue statement **skips the rest of the current loop iteration** and moves directly to the next iteration.
- When Dart encounters continue, it **jumps back to the loop condition check** without executing any code below the continue in that iteration.
- It is useful when you want to **skip certain iterations** but continue looping.

**Example use case:** Skip processing certain values based on a condition.

# Mobile App Development

## Control Flows – Break and Continue



### Example using Break

```
void main() {  
    for (int i = 10; i >= 1; i--) {  
        if (i == 7) {  
            break;  
        }  
        print(i);  
    }  
}
```

### OUTPUT:

10  
9  
8

### Example using Continue

```
void main() {  
    for (int i = 10; i >= 1; i--) {  
        if (i == 4) {  
            continue;  
        }  
        print(i);  
    }  
}
```

In Programming, loops are used to repeat a block of code until certain conditions are not completed. There are different types of loop in Dart.

- **For Loop**
- **For Each Loop**
- **While Loop**
- **Do While Loop**

In **Dart**, both **for loop** and **forEach** are used for **iterating** over collections like **lists**, but they have different syntax and use cases.

### SYNTAX

```
for (var i = 0; i < list.length; i++) {  
    print(list[i]);  
}
```

### EXAMPLE

```
void main() {  
    List<String> fruits = ['Apple', 'Banana', 'Mango'];  
  
    for (int i = 0; i < fruits.length; i++) {  
        print('Fruit $i: ${fruits[i]}');  
    }  
}
```

# Mobile App Development

## Control Flows – For Loop



```
void main() {  
    for (int i = 0; i < 10; i++) {  
        print("Hello...");  
    }  
}
```

### OUTPUT:

Hello...  
Hello...  
Hello...  
Hello...  
Hello...  
Hello...  
Hello...  
Hello...  
Hello...  
Hello...

### 2. for-in Loop (Enhanced for loop / for-each)

This is a cleaner way to loop **without using the index**.

**OUTPUT:**

#### SYNTAX

```
for (var item in list) {  
    print(item);  
}
```

#### EXAMPLE

```
void main() {  
    List<String> colors = ['Red', 'Green', 'Blue'];  
  
    for (var color in colors) {  
        print('Color: $color');  
    }  
}
```

### 3. `forEach()` Method

This is a method defined on collections like List. It uses a **callback function**.

#### **Syntax:**

```
list.forEach((element) { print(element); });
```

#### **Example:**

```
void main() { List<int> numbers = [10, 20, 30];  
numbers.forEach((num) { print('Number: $num'); }); }
```

# Mobile App Development

## Control Flows – for each Loop

---



```
void main(){  
    List<String>footballplayers=['Ronaldo','Messi','Neymar','Hazard'];  
    footballplayers.forEach( (names)=>print(names));  
}
```

### OUTPUT:

Ronaldo  
Messi  
Neymar  
Hazard



# Mobile App Development

## Control Flows – for each Loop



```
void main() {  
    List<String> items = ['Pen', 'Pencil', 'Eraser', 'Sharpener'];  
  
    print('--- Using traditional for loop ---');  
    for (int i = 0; i < items.length; i++) {  
        print('Item at index $i: ${items[i]}');  
    }  
  
    print('\n--- Using for-in loop (enhanced for loop) ---');  
    for (var item in items) {  
        print('Item: $item');  
    }  
  
    print('\n--- Using forEach loop ---');  
    items.forEach((element) {  
        print('Element: $element');  
    });  
}
```

### OUTPUT:

```
--- Using traditional  
for loop ---  
Item at index 0: Pen  
Item at index 1: Pencil  
Item at index 2: Eraser  
Item at index 3:  
Sharpener  
  
--- Using for-in loop  
(enhanced for loop) ---  
Item: Pen  
Item: Pencil  
Item: Eraser  
Item: Sharpener  
  
--- Using forEach loop  
---  
Element: Pen  
Element: Pencil  
Element: Eraser  
Element: Sharpener
```

# Mobile App Development

## Control Flows – While Loop



```
while(condition)
{
    //statement(s);
}
```

```
void main(){
    int i = 50;
    while(i<=100){
        if(i%2 == 0)
        {
            print(i);
        }
        i++;
    }
}
```

This program will print even numbers between 50 to 100 using while loop.

# Mobile App Development

## Control Flows – Do While Loop



```
do {  
    statement1;  
    statement2;  
    .  
    .  
    .  
    statementN;  
} while(condition);
```

```
void main() {  
    int i = 10;  
    do {  
        print(i);  
        i--;  
    } while (i >= 1);  
}
```

This program will print 10 to 1 in reverse order using do while loop.

# Mobile App Development

## Practice Exercise

---



1. Write a dart program to check if the number is odd or even.
2. Write a dart program to check whether a number is positive, negative, or zero.
3. Write a dart program to calculate the sum of natural numbers.
4. Write a dart program to generate multiplication tables of 7.



THANK YOU

---

**Dr. Annu Sharma**  
**Associate Professor**  
Department of Computer Science



# **MOBILE APP DEVELOPMENT**

**(UQ23CA243AA1)**

---

**Dr. Annu Sharma**  
**Associate Professor**

Department of Computer Applications

# MOBILE APP DEVELOPMENT

---

## Dart Operators

**Dr.Annu Sharma**

Department of Computer Applications

# Mobile App Development

## Dart Operators

---



- Arithmetic Operators
- Relational Operators
- Type Test Operators
- Logical Operators
- Bitwise and Shift Operators



# Mobile App Development

## Arithmetic Operators

- Arithmetic operators are commonly used on int and double to build expressions

| Symbol | Meaning                               | Example        |
|--------|---------------------------------------|----------------|
| +      | Add two values                        | $2 + 3 // 5$   |
| -      | Subtract two values                   | $2 - 3 // -1$  |
| *      | Multiply two values                   | $6 * 3 // 18$  |
| /      | Divide two values                     | $9 / 2 // 4.5$ |
| ~/     | Integer division of two values        | $9 ~/ 2 // 4$  |
| %      | Remainder (modulo) of an int division | $5 \% 2 // 1$  |

| Symbol | Meaning                  | Example |
|--------|--------------------------|---------|
| ++     | Prefix/Postfix Increment | ++2 //3 |
| --     | Prefix/Postfix Decrement | 2-- //1 |

- In the prefix version (++x) the value is first incremented and then "returned";
- In the postfix version (x++) the value is first "returned" and then incremented

- Equality and relational operators are used in Boolean expression, generally inside if statements or as a stop condition of a while loop.

# Mobile App Development

## Relational Operators

| Symbol | Meaning             | Example |
|--------|---------------------|---------|
| ==     | Equality test       | 2 == 6  |
| !=     | Inequality test     | 2 != 6  |
| >      | Greater than        | 2 > 6   |
| <      | Smaller than        | 2 < 6   |
| >=     | Greater or equal to | 2 >= 6  |
| <=     | Smaller or equal to | 2 <= 6  |

# Mobile App Development

## Type Test Operators

- They are used to check the type of an object at runtime.

| Symbol | Meaning                                | Example       |
|--------|----------------------------------------|---------------|
| as     | Cast a type to another                 | obj as String |
| is     | True if the object has a certain type  | obj is double |
| is!    | False if the object has a certain type | obj is! int   |

# Mobile App Development

## Logical Operators



| Symbol         | Meaning                                   |
|----------------|-------------------------------------------|
| !expr          | Toggles true to false and vice versa      |
| expr1 && expr2 | Logical AND (true if both sides are true) |
| expr1    expr2 | Logical OR (true if at least one is true) |

# Mobile App Development

## Bitwise and Shift Operators

| Symbol       | Meaning            |
|--------------|--------------------|
| $a \& b$     | Bitwise AND        |
| $a   b$      | Bitwise OR         |
| $a \wedge b$ | Bitwise XOR        |
| $\sim a$     | Bitwise complement |
| $a \gg b$    | Right shift        |
| $a \ll b$    | Left shift         |

# Mobile App Development

## Sample Program with Arithmetic operators



```
void main() {  
    // Arithmetic Operators  
    int a = 10;  
    int b = 3;  
  
    print('Arithmetic Operators:');  
    print('a + b = ${a + b}');           // Addition  
    print('a - b = ${a - b}');           // Subtraction  
    print('a * b = ${a * b}');           // Multiplication  
    print('a / b = ${a / b}');           // Division  
    print('a % b = ${a % b}');           // Modulus  
    print('a ~/ b = ${a ~/ b}');         // Integer Division  
  
}
```



# Mobile App Development

## Sample Program with Relational operators



```
void main() {  
    int a = 10;  
    int b = 3;  
  
    // Relational Operators  
  
    print('\n Relational Operators:');  
    print('a == b is ${a == b}'); // Equal to  
    print('a != b is ${a != b}'); // Not equal to  
    print('a > b is ${a > b}'); // Greater than  
    print('a < b is ${a < b}'); // Less than  
    print('a >= b is ${a >= b}'); // Greater than or equal to  
    print('a <= b is ${a <= b}'); // Less than or equal to  
  
}
```

# Mobile App Development

## Sample Program with Logical operators

---



```
void main() {  
  
    // Logical Operators  
    bool x = true;  
    bool y = false;  
    print('\nLogical Operators:');  
  
    print('x && y is ${x && y}');           // Logical AND  
    print('x || y is ${x || y}');          // Logical OR  
    print('!x is ${!x}');                  // Logical NOT  
  
}
```

# Mobile App Development

## Sample Program with Bitwise operators



```
void main() {  
  
    // Bitwise Operators  
    int c = 5; // 0101 in binary  
    int d = 3; // 0011 in binary  
  
    print('\n Bitwise Operators:');  
    print('c & d = ${c & d}');           // Bitwise AND  
    print('c | d = ${c | d}');           // Bitwise OR  
    print('c ^ d = ${c ^ d}');           // Bitwise XOR  
    print('~c = ${~c}');                  // Bitwise NOT  
    print('c << 1 = ${c << 1}');          // Left Shift  
    print('c >> 1 = ${c >> 1}');          // Right Shift  
  
}
```

# Mobile App Development

## Sample Program with Assignment operators



```
void main() {  
    // Assignment Operators  
    int e = 10;  
    print('\n Assignment Operators:');  
    e += 5;                // e = e + 5  
    print('e += 5 -> e = $e');  
    e -= 3;                // e = e - 3  
    print('e -= 3 -> e = $e');  
    e *= 2;                // e = e * 2  
    print('e *= 2 -> e = $e');  
    e ~/ 2;                // e = e ~/ 2  
    print('e ~/ 2 -> e = $e');  
    e %= 3;                // e = e % 3  
    print('e %= 3 -> e = $e');  
  
    int('g ?? 30 = ${g ?? 30}'); // Return g if not null, else 30  
}
```

# Mobile App Development

## Sample Program with Assignment operators

---



```
void main() {  
  
    // Conditional (Ternary) Operator  
    int f = 15;  
    String result = f > 10 ? 'Greater than 10' : '10 or less';  
    print('\n Conditional Operator:');  
    print('f is $f, so result is "$result"');  
}
```



THANK YOU

---

**Dr. Annu Sharma**  
**Associate Professor**  
Department of Computer Science



# **MOBILE APP DEVELOPMENT**

**(UQ23CA243AA1)**

---

**Dr. Annu Sharma**  
**Associate Professor**

Department of Computer Applications

# MOBILE APP DEVELOPMENT

---

## Data types – Map

**Dr. Annu Sharma**

Department of Computer Applications



- In a Map, data is stored as keys and values. In Map, each key must be unique.
- The values can be duplicated.
- Dart maps provide various methods to manipulate and interact with the data.

# Mobile App Development

## Dart Data Types – Declaring Map



```
void main() {  
  Map<String, String> myDetails = {  
    'name': 'Annu sharma',  
    'university': 'PESU',  
    'city': 'Bengaluru',  
  };  
  // displaying the output  
  print(myDetails['name']);  
}
```

**Output:**  
Annu Sharma

```
void main() {  
    // Creating a Map  
    Map<String, int> credits = {  
        'Flutter': 30,  
        'Web Services': 25,  
        'PHP': 35,  
    };  
    // Accessing values by key  
    print(credits['Flutter']); // Output: 30  
    // Adding a new key-value pair  
    credits['NoSQL'] = 40;  
    // Checking if a key exists  
    if (credits.containsKey('PHP')) {  
        print('PHP is in the map.');    }  
}
```



```
// Before deletion a key-value pair  
print(credits);  
  
// Removing a key-value pair  
credits.remove('Web Services');  
  
// After deletion a key-value pair  
print(credits);  
  
// Iterating over the map  
credits.forEach((key, value) {  
    print('$key subject holds $value credits.');
```

### Output:

30

PHP is in the map.

```
{Flutter: 30, Web Services: 25, PHP: 35, NoSQL: 40}
```

```
{Flutter: 30, PHP: 35, NoSQL: 40}
```

Flutter subject holds 30 credits.

PHP subject holds 35 credits.

NoSQL subject holds 40 credits.

- **length:** Returns the number of key-value pairs in the map.
- **isEmpty:** Returns true if the map contains no key-value pairs.
- **isNotEmpty:** Returns true if the map contains one or more key-value pairs.
- **clear():** Removes all key-value pairs from the map.
- **keys:** Returns an iterable of the map's keys.
- **values:** Returns an iterable of the map's values.

```
void main() {  
  // Creating a Map  
  Map<String, int> inventory = {  
    'Apples': 50,  
    'Bananas': 30,  
    'Oranges': 20,  
  };  
  
  // Accessing a value by key  
  print('Apples: ${inventory['Apples']}' );    // Output: Apples: 50  
  
  // Checking the length of the map  
  print('Inventory Length: ${inventory.length}');    // Output: Inventory Length: 3  
}
```



// Checking if the map is empty

```
print('Is Inventory Empty?: ${inventory.isEmpty}');
```

// Output: Is Inventory Empty?: false

// Getting all keys

```
print('Inventory Keys: ${inventory.keys}');
```

// Output: Inventory Keys: (Apples, Bananas, Grapes)

// Getting all values

```
print('Inventory Values: ${inventory.values}');
```

// Output: Inventory Values: (50, 30, 40)

```
// Iterating over the map
inventory.forEach((key, value) {
  print('$key: $value');
});
// Output:
// Apples: 50
// Bananas: 30
// Grapes: 40

// Clearing the map
inventory.clear();
print('Cleared Inventory: $inventory');    // Output: Cleared Inventory: {}
}
```

### Practice Exercise

Create a Dart map that stores the **names of students** and their corresponding **grades**.

Perform the following operations :

- Basic operations like adding, updating, and removing entries.
- Iterate over the map entries to print each student's name and their grade.
- Check if certain keys or values exist in the map.



THANK YOU

---

**Dr. Annu Sharma**  
**Associate Professor**  
Department of Computer Science



# **MOBILE APP DEVELOPMENT**

**(UQ23CA243AA1)**

---

**Dr. Annu Sharma**  
**Associate Professor**

Department of Computer Applications

# MOBILE APP DEVELOPMENT

---

## Functions and Structures

Dr. Annu Sharma

Department of Computer Applications

- Dart is a true object-oriented language, so even functions are objects and have a type,
- The functions can be assigned to variables or passed as arguments to other functions.
- Dart function is a set of codes that together perform a specific task.
- It is used to break the large code into smaller modules and reuse it when needed.

Functions make the program more readable and easier to debug.

- It improves the modular approach and enhances the code reusability.

### Defining the Function in Dart

Dart provides us with the facility of using functions in its program.

```
return_type function_name ( parameters )  
{  
    // Body of function  
    return value;  
}
```

In the above syntax:

- **function\_name:** defines the name of the function.
- **return\_type:** defines the datatype in which output is going to come.
- **return value:** defines the value to be returned from the function.



## Function Call in Dart

```
function_name ( argument_list )
```

In the above syntax:

- **function\_name:** defines the name of the function.
- **argument list:** is the list of the parameters that the function requires.

```
int mul(int a, int b){  
    // Creating function  
    int result = a * b;  
    // returning value result  
    return result;  
}  
  
void main(){  
    // Calling the function  
    var output = mul(10, 20);  
  
    // Printing output  
    print(output);  
}
```

**OUTPUT**

**200**

# Mobile App Development

## Functions Parameters and Return Types

---



**Example : Function without parameter and return value.**

```
void sample(){  
    // Creating function  
    print("Welcome to PES");  
}
```

```
void main()  
{  
    // Calling the function  
    sample();  
}
```

**Example : Function with parameter and return value.**

```
bool checkEven(int value) {  
    return value % 2 == 0;  
}  
void main() {  
    print(checkEven(19));  
}
```

### Function with parameter and return value.

- When the body of the function contains only one line, omit the braces and the return statement in favor of the "arrow syntax".
- It works with expressions and not with statements

### Function with parameter and return value.

```
bool checkEven(int value) => value % 2 == 0;
void main()
{
    print(checkEven(10));
}
```

### Function

```
bool checkEven(int value) => value  
% 2 == 0;
```

```
void main()  
{  
print(checkEven(10));  
}
```

**Writing a type (bool Function(int)), its name (checker) and then assigning it a value (checkEven)**

```
bool checkEven(int value) => value  
% 2 == 0; // Declare a function
```

```
void main() {  
    bool    Function(int)    checker    =  
    checkEven; // Assign a function to a variable
```

```
    print(checker(8));    //    Use the variable that represents the function
```



THANK YOU

---

**Dr. Annu Sharma**  
**Associate Professor**  
Department of Computer Science





# MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

---

**Dr. Annu Sharma**  
**Associate Professor**

Department of Computer Applications

# MOBILE APP DEVELOPMENT

---

## Functions and Structures

Dr. Annu Sharma

Department of Computer Applications

### Parameters

A function can have any number of required positional parameters. These can be followed either by named parameters or by optional positional parameters (but not both).

The Dart Functions Parameters can be categorized into two types :—

- **Named** : The name of an argument determines which parameter receives the value.
- **Positional**-The position of an argument determines which parameter receives the value.

**Positional:** The position of an argument determines which parameter receives the value.

**EXAMPLE:**

```
void main() {  
    void add(a, b) { // a & b are positional parameters  
        print(a + b); // print() is a built-in function that will be explained later  
    }  
    add(5, 10);  
    // 5 is used as a value for a, because it's the first argument; 10 is used as a value for b.  
  
}
```

### Named parameters

- Named parameters are optional unless they're explicitly marked as required.
- When defining a function, use {param1, param2, ...} to specify named parameters.
- The parameters' name must be specified while the value is being passed. Curly brace {} can be used to specify optional named parameters.
- If we don't provide a default value or mark a named parameter as required, their types must be nullable as their default value will be null:

### **EXAMPLE : Named parameters**

```
void main() {  
    void add({a, b})  
    { // a & b are named parameters (because of the curly braces)  
        print(a + b);  
    }  
}
```

add(b: 5, a: 10); // 5 is used as a value for b, because it's assigned to that name; 10 is used as a value for a.

```
}
```

### Functions with Optional Parameter

There are also optional parameter system in Dart which allows user to give some optional parameters inside the function.

| Sr. No. | Parameter                              | Description                                                                                                          |
|---------|----------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| 1.      | Optional Positional Parameter          | To specify it use square ('[ ]') brackets                                                                            |
| 2.      | Optional Named parameter               | When we pass this parameter it is mandatory to pass it while passing values. It is specify by curly('{ }') brackets. |
| 3.      | Optional parameter with default values | Here parameters are assign with default values.                                                                      |

### Optional Named Parameters in DART

#### Syntax - Declaring the function

```
void function_name(a, {optional_param1, optional_param2}) { }
```

#### Syntax - Calling the function

```
function_name(optional_param:value,...);
```



### Named parameters

```
void main() {  
    nmp(123);  
    nmp(123,s1:'hello');  
    nmp(123,s2:'PES',s1:'world');  
}  
nmp(n1,{s1,s2}) {  
    print(n1);  
    print(s1);  
    print(s2);  
}
```

### Output

```
123  
null  
null  
123  
hello  
null  
123  
world  
PES
```

```
void fun1(int g1, [ var g2 ])
{
    // Creating function 1
    print("g1 is $g1");
    print("g2 is $g2");
}

void fun2(int g1, { var g2, var g3 })
{
    // Creating function 1
    print("g1 is $g1");
    print("g2 is $g2");
    print("g3 is $g3");
}
```

```
void fun3(int g1, { int g2 = 12 })
{ print("g1 is $g1");
  print("g2 is $g2");
}

void main(){
    print("Calling the function with
optional parameter:");
    fun1(01);
    print("Calling the function with
Optional Named parameter:");
    fun2(01, g3 : 12);
    print("Calling function with default
valued parameter");
    fun3(01);
}
```

In addition to their distinct uses, positional and named arguments differ in one crucial way:

Positional parameters are by default necessary and cannot be omitted, but named arguments are optional and can be omitted.

Dart gives the possibility to create nameless functions called **anonymous functions**.

- An anonymous function in Dart is like a named function but they do not have names associated with it.
- An anonymous function can have zero or more parameters with optional type annotations.
- An anonymous function consists of self-contained blocks of code and that can be passed around in our code as a function parameter.

# Mobile App Development

## Dart – Anonymous Functions



### Syntax:

```
(parameter_list)

{

    statement(s)

}
```

### Example

```
void main()
{
    var list = ["Shubham","Nick","Adil","Puthal"];
    print("Anonymous function in Dart");
    list.forEach((item) {
        print('$ {list.indexOf(item)} : $item');
    });
}
```

# Mobile App Development

## Dart – Anonymous Functions

---



- The example defines an anonymous function with an untyped parameter, item. The function, invoked for each item in the list, prints a string that includes the value at the specified index.
- In Dart we can also create nameless function known as an anonymous function, lambda, or closure.
- we can assign an anonymous function to constants or variables, later we can access or retrieve the value of closure based on our requirements.

- This syntax allows creation of functions "on the fly" that are immediately assigned to a variable.
- If there is a need for anonymous function with no parameters, leave the parenthesis blank ().
- final and var can be used to automatically deduce the type

### Anonymous Functions

- **Single line**

**Use the arrow syntax.**

- This example declares a function with no parameters that returns a double.

```
final anon = () => print(5.8 + 12);  
void main() { anon(); }
```



### Multiple lines

Use brackets and return when there is a need to implement a logic that's longer than one line.

### Example

```
final anon = (String nickname) {  
  var myName = 'PES Academy';  
  myName += nickname;  
  return myName;  
};  
• void main() {  
• var res = anon('Hello'); print(res); }
```

## Dart – Using Action in functions

The action parameter commonly known as callback because it executes an action given from the outside.

### Examples

```
void test(void Function(int) action) { //1
    final list = [1, 2, 3, 4, 5]; // 2
    for(final item in list) // 3
        action(item); }
void main() {
    test(// 4
        (int value) { print('Number $value'); } // 5
    );
}
```

- .. This function doesn't return a value because of the void.
  - The parameter, called action, accepts a void function with a single integer value.
- !. It's a simple list of integer values
- }. Iterate through the entire list and, for each item, call the function.



THANK YOU

---

**Dr. Annu Sharma**  
**Associate Professor**  
Department of Computer Science

**PES University**  
Department of Computer Applications – RR Campus  
**BCA-III Semester**  
**UQ23CA243AA1 - Mobile App Development**  
Assignment – UNIT - 1

---

**SECTION A: Experiential Learning**

**Instructions :** Dear Students, kindly solve and execute following programs. For submission, create a pdf file and for each problem mention the problem statement, code and the output generated (screenshot). For theory Question write all the question in Blue book, scan it create pdf and upload it .

- 1. Multiplication Table Generator :** Write a program that generates the multiplication table for a given number. Perform using looping, conditionals statements and functions. Provide proper validations wherever required. Comment on the code with proper naming convention to explain the loop and output.
  
- 2. Mathematical operations :** Write a program to enter a number. Perform following operations using looping, conditionals statements and functions:
  - a. Check whether the number is Positive, Negative or Zero.
  - b. Check whether the number is Even / Odd.
  - c. Check whether the number is Prime / Non – Prime.
  - d. Calculate factorial for the given number.
  - e. Take 3 digit number and perform following operations:
    - Sum of digits for the given number
    - Reverse the number
  
- 3. List operations :** Create a list of integers and perform following operations as described below :
  - a. Write a function that removes duplicate elements from a list of integers.
  - b. Write a function that print the list in a reverse order.
  - c. Write a function that returns a new list containing only the unique elements.
  - d. Write a function to count the duplicated values
  
- 4. Map operations :** Create a simple banking system with features like deposit, withdraw, and check balance using Maps in flutter. Provide proper validations, comments.

**PES University**  
Department of Computer Applications – RR Campus  
**BCA-III Semester**  
**UQ23CA243AA1 - Mobile App Development**  
Assignment – UNIT - 1

---

- 5. Set operations :** Create two sets and perform all set operations with proper outputs.

**PES University**  
Department of Computer Applications – RR Campus  
**BCA-III Semester**  
**UQ23CA243AA1 - Mobile App Development**  
Assignment – UNIT - 1

---

**SECTION B: THEORY EXERCISE**

6. Define Dart programming and what are its features?
7. What are the data types in the Dart language? Explain Set and Map with examples.
8. Explain the conditional operator (also known as the ternary operator) in Dart. How does it work, and in what scenarios would you use it? Provide an example to illustrate your explanation.
9. Discuss the different types of loops available in Dart. How do they work, Provide examples to illustrate the usage of for each loop type.
10. Explain the concept of functions in Dart. What are the different ways to define and use functions? Discuss the significance of named parameters, optional parameters, and anonymous functions in Dart, providing examples for each.